Trabalho Prático - Análise Empírica de Complexidade de Algoritmos

Matheus Miguel Soares de Medeiros Lima

1 Introdução

A avaliação da eficiência de algoritmos é essencial para prever o comportamento e a viabilidade de programas à medida que o tamanho da entrada (N) cresce. A análise assintótica, frequentemente expressa por meio da notação Big-O, fornece um modelo matemático baseado na taxa de crescimento em tempo de execução, estabelecendo um limite superior para o comportamento do algoritmo para um N suficientemente grande. Matematicamente, diz-se que uma função de tempo $f(n)$ é $O(g(n))$ se existem constantes positivas c e n_0 tais que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$.

Embora a análise teórica seja necessária, a análise empírica é uma das formas de se compreender o impacto das constantes ocultas (c) e dos fatores físicos da máquina (como o uso de memória cache e o escalonamento do sistema operacional) no tempo de execução real.

O objetivo deste trabalho é implementar, medir e analisar empiricamente o tempo de execução de algoritmos clássicos, comparando o comportamento dos dados coletados com as suas respectivas complexidades teóricas. Para este estudo, foram selecionados dois algoritmos de busca e dois de ordenação, de forma a comparar se as curvas de tempo geradas pelo experimento correspondem às curvas teóricas.

2 Descrição dos Algoritmos

Para o experimento, foram selecionados e implementados quatro algoritmos, descritos a seguir:

2.1 Algoritmos de Busca

- **Busca Sequencial ($O(n)$):** Consiste em uma iteração linear simples sobre os elementos de uma estrutura de dados, verificando um a um se o elemento atual corresponde à chave buscada. No pior caso (quando o elemento não existe ou é o último da lista), exige a verificação de todo o conjunto de tamanho N, resultando em uma complexidade de tempo de ordem linear.

- **Busca Binária ($O(\log n)$):** Baseada no paradigma de divisão e conquista, exige que o vetor de entrada esteja previamente ordenado. Esse algoritmo compara a chave procurada com o elemento central do vetor, descartando metade do espaço de busca a cada iteração recursiva. Devido a esta divisão sucessiva, sua complexidade no pior caso é de crescimento logarítmico, tornando-o altamente eficiente para grandes volumes de dados.

2.2 Algoritmos de Ordenação

- **MergeSort ($O(n \log n)$):** Baseado na divisão e conquista. Divide o vetor ao meio de forma recursiva em subvetores, até que cada um contenha apenas um elemento. Em seguida, compara os elementos dos subvetores e os mescla de forma a construir arranjos ordenados. Enquanto apresenta complexidade de $O(n \log n)$ em qualquer cenário, apresenta a desvantagem de requerer um espaço suplementar $O(n)$ para os subvetores auxiliares.

- **QuickSort ($O(n \log n)$):** Também baseado em divisão e conquista, este algoritmo seleciona um elemento designado como "pivô"e particiona o vetor: elementos menores à esquerda do pivô e elementos maiores à direita. A ordenação é feita de forma recursiva em cada partição. Embora possua um pior caso teórico de complexidade quadrática $O(n^2)$, apresenta uma complexidade média log-linear $\Theta(n \log n)$, dispensando alocações expressivas de memória e tendo alta velocidade de execução.

3 Metodologia

O projeto foi desenvolvido em linguagem C++ (padrão C++17), estruturado de forma modular para desacoplar a lógica dos algoritmos das funções de medição e exportação de dados. O tempo de execução foi contabilizado utilizando a biblioteca nativa `<chrono>`, empregando o `high_resolution_clock` para garantir a precisão das medições em nível de nanossegundos.

Medições empíricas de *software* podem conter ruídos, uma vez que o escalonador do Sistema Operacional pode interromper a execução para processar outras rotinas em *background*. Para mitigar essas anomalias e obter um tempo de execução médio estatisticamente confiável, foi feito o seguinte planejamento amostral:

- **Buscas:** Para cada tamanho de entrada N, a execução foi repetida 1000 vezes, calculando a média de tempo. O vetor de testes foi previamente ordenado e a chave de busca foi propositalmente definida como um elemento inexistente no conjunto (-1). Dessa forma garantimos a ocorrência do *pior caso* algorítmico.

- **Ordenações:** Por ter um maior custo computacional, a execução foi repetida 10 vezes para cada tamanho N, calculando-se a média. Os vetores de entrada foram gerados em ordem decrescente. Para o QuickSort, a escolha do pivô foi implementada utilizando o elemento central do subvetor, mitigando o risco de colisão com o seu pior caso teórico clássico.

Ao término das execuções, os pares ordenados $(N, \text{tempo médio})$ foram exportados para um arquivo CSV. Foi executado um *script* em Python para confeccionar os gráficos Tamanho da Entrada x Tempo de Execução, utilizando as bibliotecas `pandas`, `matplotlib` e `scipy.optimize` (`curve_fit`).

4 Resultados

A seguir, são apresentados os gráficos resultantes da análise empírica. As medições em pontos representam os tempos obtidos em C++, enquanto as linhas contínuas representam as funções teóricas ajustadas.

4.1 Algoritmos de Busca

Para a plotagem das buscas, o eixo das ordenadas (Tempo) foi configurado em escala logarítmica, para acomodar visualmente a grande diferença entre o crescimento linear e o crescimento sublinear.

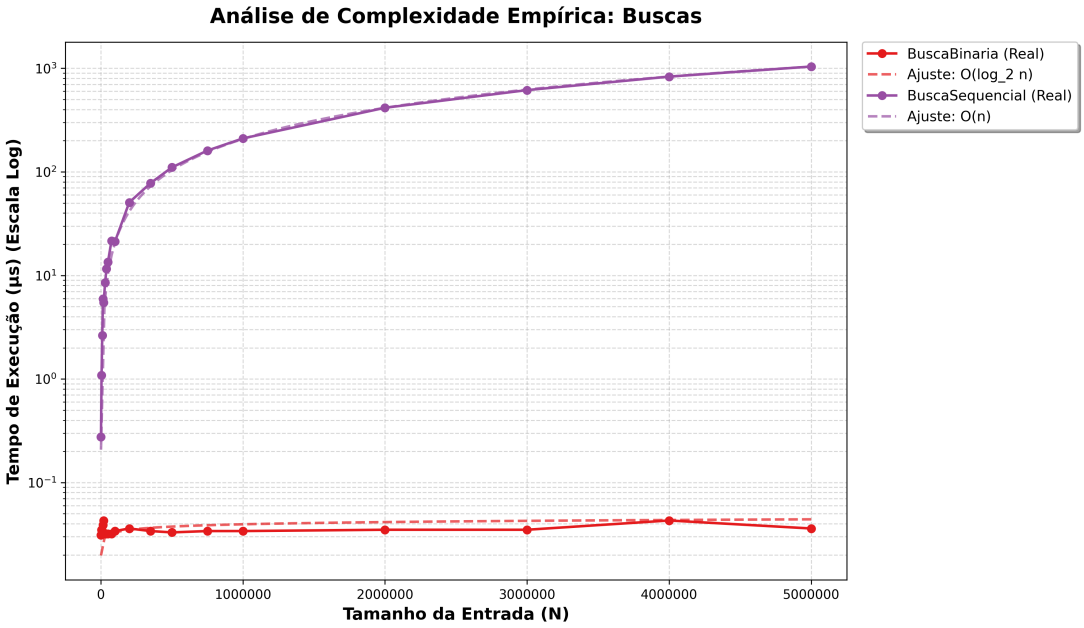

Figura 1: Tempo de Execução vs Tamanho da Entrada para Algoritmos de Busca.

Conforme ilustrado na Figura 1, a Busca Sequencial exibe um crescimento linear, ficando muito próxima da curva teórica $O(n)$. Em contrapartida, a Busca Binária mostra a velocidade dos algoritmos do tipo "divisão e conquista": mesmo para um N cinco milhões, o tempo gasto é de microfrações de segundo, confirmando de forma empírica a estabilidade do comportamento $O(\log n)$.

4.2 Algoritmos de Ordenação

A Figura 2 compara os desempenhos práticos do MergeSort e do QuickSort.

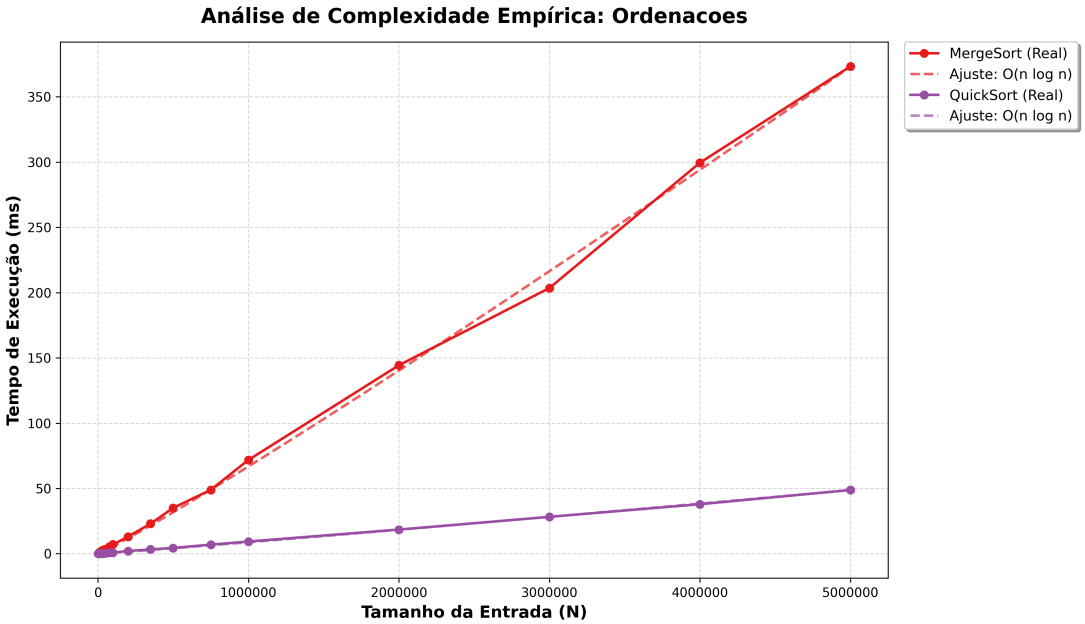

Figura 2: Tempo de Execução vs Tamanho da Entrada para Algoritmos de Ordenação.

Ambos os algoritmos pertencem à mesma classe de complexidade média, $\Theta(n \log n)$, o que é atestado pelas curvas assintóticas dos dados coletados. No entanto, o QuickSort apresenta um tempo de execução real mais rápido que o MergeSort.

Este resultado denota a importância da análise da "constante oculta". O MergeSort exige a alocação, cópia e liberação ininterrupta de memória dinâmica a cada operação de mescla. Enquanto o QuickSort manipula as posições do próprio vetor original em tempo de execução, reduzindo o custo que operações recorrentes de gestão de memória exercem sobre a arquitetura do computador.

5 Conclusão

Os experimentos empíricos realizados concordam com os princípios teóricos da análise de complexidade. A adoção de uma metodologia sólida — baseada em amostragem repetida para diminuir as interferências de sistema — garantiu o desenho de curvas fiéis aos comportamentos $O(n)$, $O(\log n)$ e $O(n \log n)$.

Conclui-se que a notação Big-O atua como bússola para medir a escalabilidade assintótica de um *software*. Porém, em cenários reais, características ligadas a arquitetura dos algoritmos — tais como as repetidas operações de memória do MergeSort, quando comparado ao QuickSort — são refletidas nos custos práticos e acabam sendo critérios de desempate importantes durante a escolha do algoritmo ideal.

Disponibilidade do Código

O código-fonte desenvolvido para este projeto, juntamente com os scripts python de plotagem dos gráficos, encontra-se documentado e publicamente disponível no GitHub. Pode ser acessado através do link: AnaliseAlgoritmica.

Referências Bibliográficas

[1] CORMEN, Thomas H. et al. *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

[2] BRASSARD, Gilles; BRATLEY, Paul. *Fundamentals of Algorithmics*. 1. ed. Englewood Cliffs: Prentice Hall, 1996.